Mini-guía

Cómo trabajar como colaborador en GitHub

Ramas, commits, push y pull requests



Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub

Nivel: Principiante – Intermedio Herramientas: Git, GitHub y Visual Studio Code Objetivo: Aprender a trabajar en un proyecto compartido sin sobrescribir el trabajo de otros compañeros.

1. Entender cómo funciona el trabajo en equipo

Cuando eres colaborador de un repositorio de GitHub, normalmente puedes trabajar directamente con el repositorio original, sin necesidad de hacer un fork.

Pero eso no significa que debas modificar directamente la rama principal (`main`).

La forma recomendada de trabajar es:

Cada colaborador trabaja en su propia rama. Los cambios se revisan antes de incorporarlos a main.

Los conceptos importantes son:

Concepto	Significado
Repositorio	Proyecto completo
`main`	Rama principal del proyecto
`branch`	Rama para trabajar sin afectar a otros
`commit`	Guardar una versión de tus cambios
`push`	Subir tus commits a GitHub
`pull`	Descargar e integrar cambios remotos
`Pull Request`	Solicitud para integrar tu trabajo
`merge`	Unir los cambios de dos ramas

Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub

2. Aceptar la invitación al repositorio

El propietario debe invitarte como colaborador.

1. Abre la invitación que recibas por correo o en GitHub.
2. Pulsa Accept invitation.
3. Accede al repositorio.
4. Comprueba que tienes permisos para trabajar.

Por ejemplo, el proyecto podría estar en:

`https://github.com/empresa/proyecto-web`

3. Clonar el repositorio en tu ordenador

Clonar significa descargar una copia del proyecto para poder trabajar localmente.

Abre la terminal de Visual Studio Code y ejecuta:

git clone https://github.com/empresa/proyecto-web.git

Entra en la carpeta:

cd proyecto-web

Abre el proyecto en VS Code:

code .

Ahora tienes el proyecto en tu ordenador.

Importante: Si ya tienes el repositorio clonado, no necesitas volver a clonarlo cada vez que trabajes.

Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub

4. Actualizar la rama principal antes de empezar

Antes de crear una funcionalidad nueva, asegúrate de tener el código más reciente del equipo.

Primero, cambia a `main`:

git switch main

Después, actualízala:

git pull origin main

Esto incorpora los cambios recientes del repositorio remoto a tu rama local.

No deberías tener modificaciones sin guardar mediante commit al realizar este paso.

5. Crear tu propia rama

Esta es una de las reglas más importantes del trabajo colaborativo.

No trabajes directamente en `main`, salvo que el equipo haya establecido expresamente ese procedimiento.

Por ejemplo, imagina que debes crear una pantalla de inicio de sesión.

Ejecuta:

git switch -c feature/login

Este comando crea una rama llamada `feature/login` y te sitúa en ella.

Para comprobar tu rama actual:

git branch

Verás algo parecido a:

* feature/login
  main

El asterisco indica tu rama actual.

Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub

@mihifife m 06/26

1802_M01

Ducky Academy

Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub

Algunos nombres habituales de ramas son:

- `feature/login`: crear una funcionalidad.
- `fix/error-menu`: corregir un error.
- `docs/readme`: modificar documentación.
- `refactor/usuarios`: reorganizar código.

6. Realizar tus modificaciones

Ahora puedes trabajar normalmente en Visual Studio Code.

Por ejemplo, podrías crear:

```
proyecto-web/
├── src/
│   ├── components/
│   │   ├── Navbar.jsx
│   │   └── Login.jsx    ← Archivo nuevo
│   ├── App.jsx
│   └── main.jsx
├── package.json
└── README.md
```

Cuando termines una parte del trabajo, comprueba qué archivos has cambiado:

git status

También puedes ver las modificaciones exactas:

git diff

Antes de subir cambios, comprueba que el proyecto sigue funcionando y ejecuta las pruebas disponibles.

Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub

7. Guardar tus cambios mediante commits

Cuando hayas terminado una parte concreta de tu tarea, debes crear un commit.

Primero añade los archivos:

git add .

Después crea el commit:

git commit -m "feat: añadir formulario de login"

`git add .` prepara todos los cambios de la carpeta actual. Si solo quieres incluir determinados archivos, puedes hacer:

git add src/components/Login.jsx

Un buen commit debería describir claramente qué has realizado.

Por ejemplo:

git commit -m "fix: corregir validación del formulario"

Consejo: Crea commits pequeños y relacionados con una sola tarea, en lugar de acumular muchos cambios diferentes en un único commit.

8. Subir tu rama a GitHub

Cuando tengas tus commits preparados:

git push -u origin feature/login

Esto publica tu rama en GitHub y configura su seguimiento remoto.

A partir de ese momento, para subir nuevos commits a la misma rama normalmente bastará con:

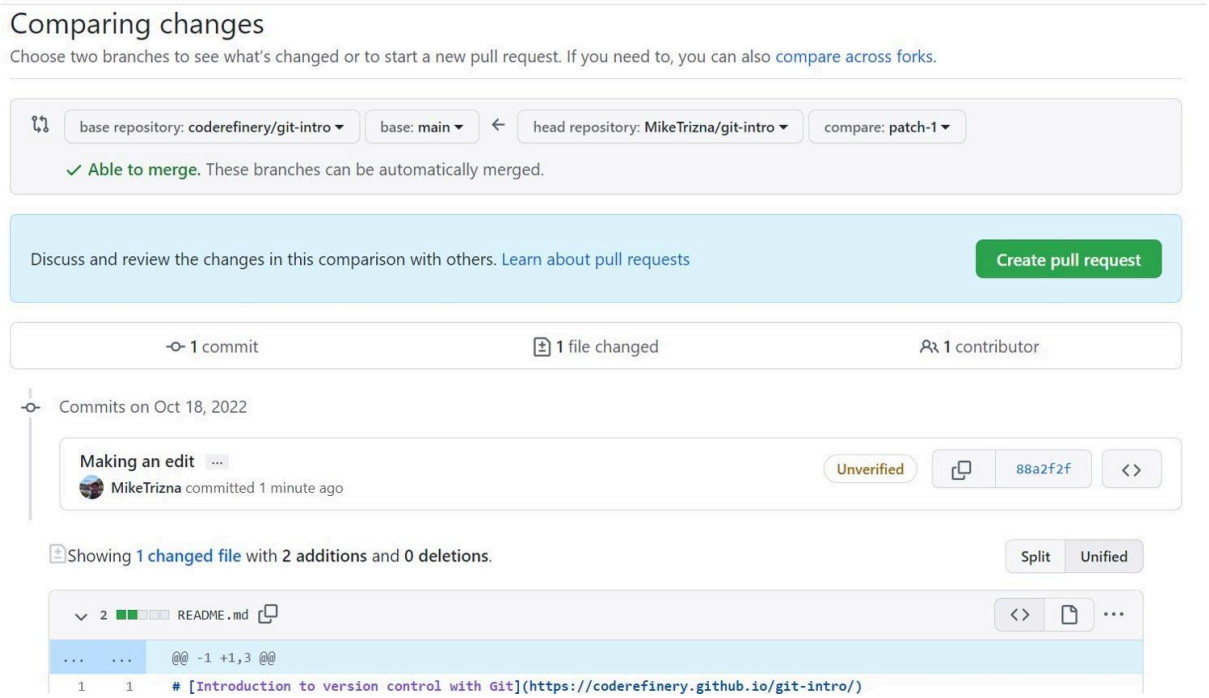
git push

En este punto, tus cambios están en GitHub, pero todavía no forman parte de `main`.

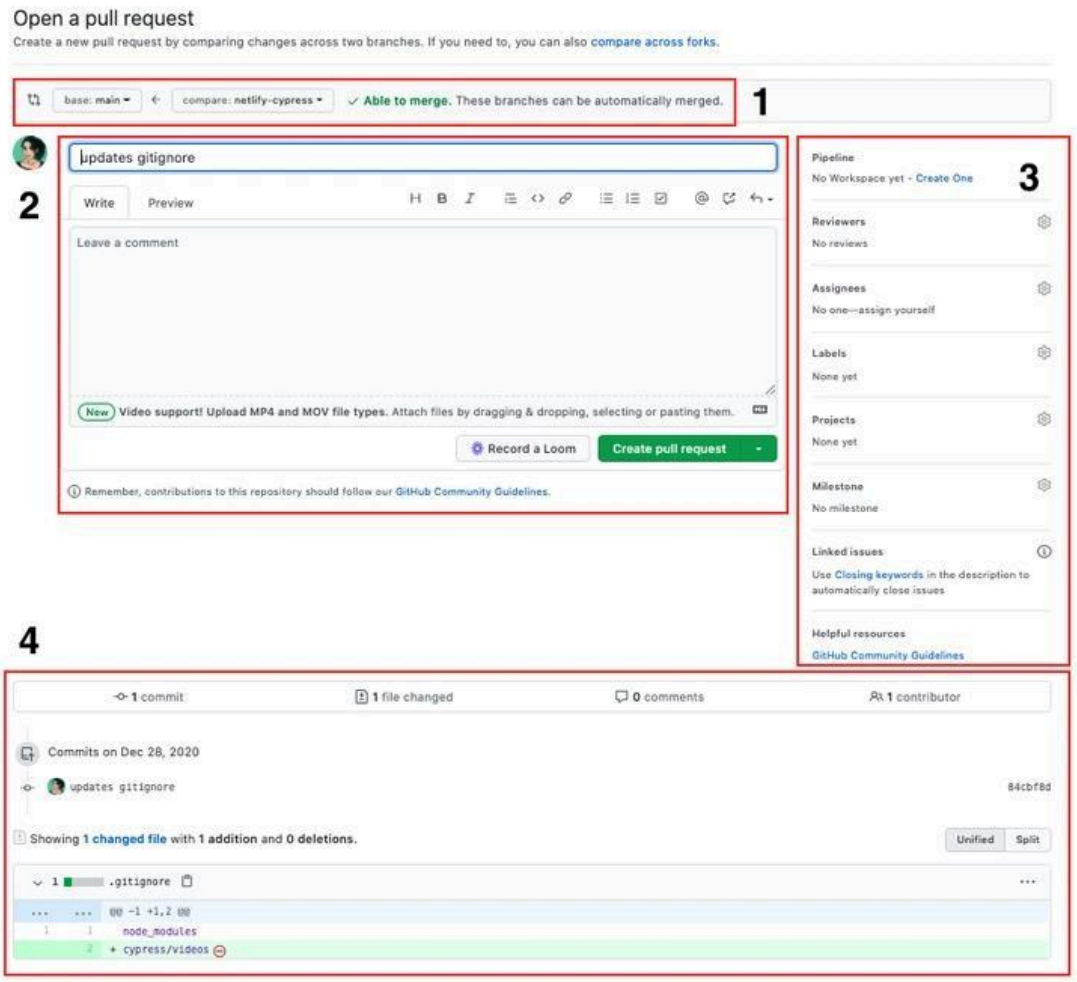
Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub

9. Crear un Pull Request

Un Pull Request (PR) es una solicitud para que otros compañeros revisen tus cambios antes de integrarlos.



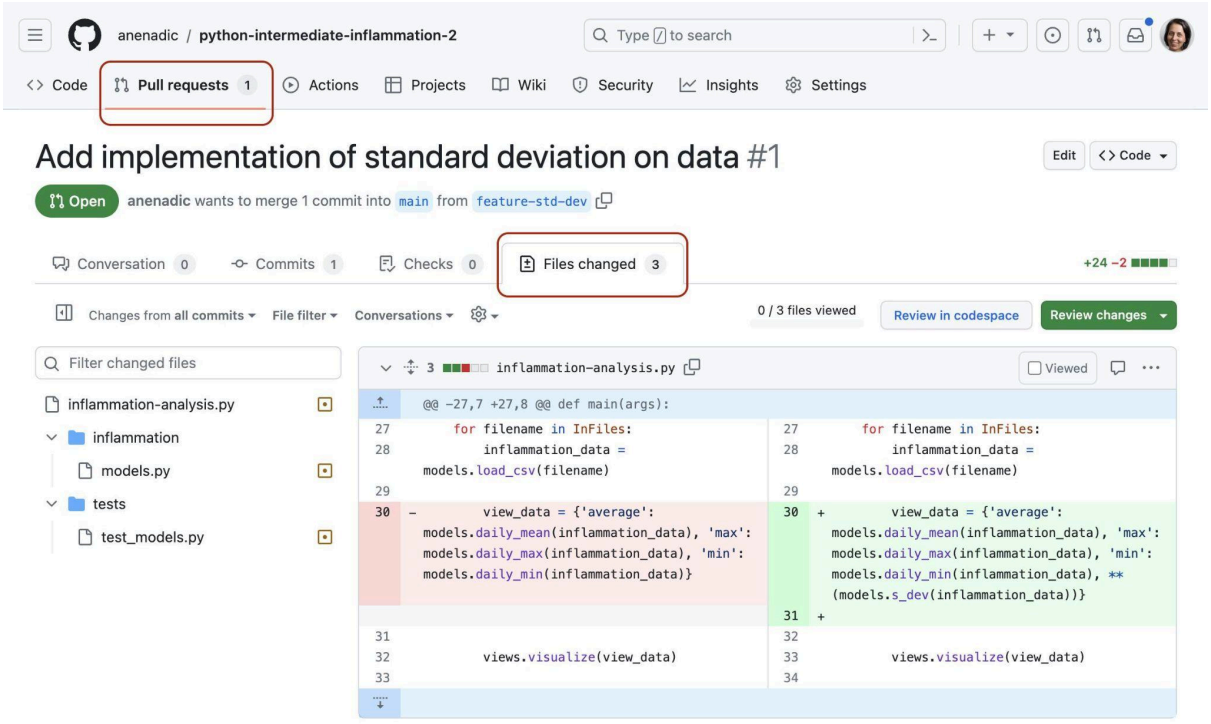
Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub



@mihififem 06/26

1802_M01

Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub



7

En GitHub:

1. Abre el repositorio.
2. Entra en Pull requests.
3. Selecciona New pull request (o Compare & pull request si aparece).
4. Selecciona `main` como rama de destino (base).
5. Selecciona `feature/login` como rama de origen (compare).
6. Escribe un título y una descripción.
7. Pulsa Create pull request.

Por ejemplo, una buena descripción sería:

Título: Añadir formulario de inicio de sesión

Descripción:

Se ha implementado el formulario de login del proyecto.

Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub

Cambios realizados:

- Nuevo componente `Login.jsx`.
- Validación de campos obligatorios.
- Mensajes de error.

Pruebas:

- Comprobado el formulario con datos válidos.
- Verificado el comportamiento con campos vacíos.

Un compañero podrá revisar tu código, pedir modificaciones o aprobarlo.

Algunos repositorios exigen revisiones y comprobaciones automáticas antes de permitir el merge.

10. Corregir cambios que te piden en el Pull Request

Imagina que un compañero revisa tu PR y dice:

> Falta validar que el correo tenga un formato correcto.

No necesitas crear otro Pull Request.

Vuelves a tu rama:

git switch feature/login

Corriges el código y ejecutas:

```
git add .
git commit -m "fix: validar formato del correo"
git push
```

Tu Pull Request se actualizará automáticamente con el nuevo commit.

Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub

11. Integrar tus cambios en main

Cuando tu Pull Request esté aprobado y cumpla las reglas del repositorio, podrá realizarse el merge.

Según los permisos y las normas del equipo, lo harás tú o una persona responsable del proyecto.

Después del merge, tu funcionalidad ya estará incorporada a la rama principal.

Para actualizar tu copia local:

git switch main
git pull origin main

Puedes eliminar tu antigua rama local si ya no la necesitas:

git branch -d feature/login

Y volver a empezar otra tarea creando una rama nueva desde `main`.

12. ¿Qué hago si mis compañeros cambian main mientras estoy trabajando?

Esto es muy habitual en los proyectos colaborativos.

Imagina esta situación:

- Tú estás trabajando en `feature/login`.
- Otro compañero termina el menú de navegación.
- Su PR se integra en `main`.
- Tu rama sigue con el código anterior.

Para incorporar esos cambios a tu trabajo, puedes hacer:

git switch main
git pull origin main
git switch feature/login
git merge main

Esto integra la versión actualizada de `main` en tu rama.

Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub

Si no existen conflictos, podrás continuar trabajando normalmente.

Si aparecen conflictos, Git te avisará y tendrás que resolverlos.

Ejemplo de conflicto

Dos personas modifican las mismas líneas de `App.jsx`.

Git podría mostrar:

<<<<<<< HEAD
<h1>Bienvenido a mi aplicación</h1>
=======
<h1>Bienvenido al proyecto</h1>
>>>>>>> main

Debes decidir qué código conservar o cómo combinar ambos cambios.

Por ejemplo:

<h1>Bienvenido al proyecto</h1>

Una vez resuelto el conflicto:

git add .
git commit
git push

El commit finaliza el merge, cuando Git lo requiere.

Atención: No elimines cambios de tus compañeros sin entenderlos. Si hay dudas, acordad la solución antes de resolver el conflicto.

Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub

13. Flujo de trabajo completo de un colaborador

Este es el procedimiento que puedes seguir cada vez que recibas una tarea nueva.

Checklist de trabajo diario

0 de 10 pasos completados

Reiniciar
Entrar en main
Descargar los últimos cambios
Crear una rama nueva para mi tarea
Programar y probar mis cambios
Revisar los archivos modificados
Crear un commit
Subir mi rama a GitHub
Abrir un Pull Request
Resolver comentarios de revisión
Actualizar main después del merge

Comandos que usarás con más frecuencia

Plantilla para una tarea nueva

Copiar

```
# 1. Actualizar main
git switch main
git pull origin main

# 2. Crear tu rama
git switch -c feature/mi-tarea

# 3. Trabajar en el código

# 4. Guardar tus cambios
git status
git add .
git commit -m "feat: describir mi cambio"

# 5. Subir la rama
git push -u origin feature/mi-tarea
```

Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub

Después del push, crea el Pull Request desde GitHub.

14. Errores que debes evitar como colaborador

Error	Buena práctica
Programar directamente en `main`	Crear una rama por tarea
Usar `git push --force` sin entenderlo	Evitar sobrescribir el historial compartido
Trabajar sin actualizar el proyecto	Hacer `git pull` al empezar
Subir cambios sin comprobarlos	Revisar `git status`, el diff y las pruebas
Mezclar tareas diferentes	Crear ramas y PR independientes
Subir contraseñas o archivos `.env` con secretos	Mantener credenciales fuera de Git
Hacer merge sin revisión	Seguir las reglas del equipo

Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub

15. Ejemplo de colaboración entre tres desarrolladores

Imagina que estáis desarrollando una página web entre tres personas.

Persona	Tarea	Rama
Desarrollador A	Crear login	`feature/login`
Desarrollador B	Crear menú	`feature/navbar`
Desarrollador C	Crear perfil de usuario	`feature/perfil`

Cada persona trabaja en su rama, hace sus commits y crea su Pull Request.

Cuando se aprueban e integran los cambios, todos actualizan su `main` antes de crear nuevas ramas.

Así podéis desarrollar funcionalidades en paralelo sin modificar directamente la rama principal.

Resumen: la regla de oro

El flujo que deberías memorizar es:

Actualizar `main` → Crear rama → Programar → Commit → Push → Pull Request → Revisión → Merge → Actualizar `main`.

Como colaborador con permisos de escritura, normalmente puedes trabajar en ramas del mismo repositorio. No necesitas hacer un fork salvo que el equipo lo exija.

Para profundizar, puedes consultar la documentación oficial de Pull Requests de GitHub.

Tutorial: Cómo trabajar como colaborador en un proyecto de GitHub

Si trabajas con un equipo de compañeros, te recomendaría utilizar una rama por tarea y un Pull Request por rama, incluso aunque GitHub os permita hacer cambios directamente en `main`. Es una forma sencilla de evitar conflictos y mantener el código organizado.

@mihififem 06/26